

Instruções Nativas		Instruções Virtuais		DETI-UA - ACI	
Transferência Memória-Registro (<i>Load</i>)		Cálculo c/ Inteiros: Operações Aritméticas		Transferência Memória-Registro (<i>Load</i>)	
lb	Rdst, addr	add	Rdst, Rsrc1, Rsrc2	l.d	FPdst, addr
lbu	Rdst, addr	addi	Rdst, Rsrc, Imm	l.s	FPdst, addr
lw	Rdst, addr	addiu	Rdst, Rsrc, Imm	Transferência Registro-Memória (<i>Store</i>)	
lwc1	CReg, addr	addu	Rdst, Rsrc1, Rsrc2	s.d	FPsrc, addr
ldc1	CReg, addr	div	Rsrc1, Rsrc2	s.s	FPsrc, addr
Transferência Registro-Memória (<i>Store</i>)		divu	Rsrc1, Rsrc2	Transferência Registro-Registro (<i>Move</i>)	
sb	Rsrc, addr	mult	Rsrc1, Rsrc2		
sw	Rsrc, addr	multu	Rsrc1, Rsrc2		
swc1	Creg, addr	sub	Rdst, Rsrc1, Rsrc2	Manipulação de Const. (<i>Load Imm/sym</i>)	
sdc1	Creg, addr	subu	Rdst, Rsrc1, Rsrc2		
Transferência Registro-Registro (<i>Move</i>)		Cálculo c/ Inteiros: Op. Lógicas Bitwise		la	Rdst, sym 
mfhi	Rdst	and	Rdst, Rsrc1, Rsrc2	li	Rdst, IMM 
mflo	Rdst	andi	Rdst, Rsrc, Imm	Cálculo c/ Inteiros: Op. Aritméticas	
mthi	Rsrc	nor	Rdst, Rsrc1, Rsrc2		
mtlo	Rsrc	or	Rdst, Rsrc1, Rsrc2		
mfcl	Rdst, Creg	ori	Rdst, Rsrc, Imm	Tabela I: Registos do MIPS e convenção de uso	
mtcl	Rsrc, Creg	xor	Rdst, Rsrc1, Rsrc2	Nome Lóg.	Nome Real
mov.d	FPdst, FPsrc	xori	Rdst, Rsrc, Imm	Uso Convencionado	
mov.s	FPdst, FPsrc	Cálculo c/ Inteiros: Operações de Shift		\$zero	\$0
Manipulação de Const. (<i>Load Immediate</i>)		sll	Rdst, Rsrc1, Imm5	\$at	\$1
		sllv	Rdst, Rsrc1, Rsrc2	\$v0..\$v1	\$2..\$3
Instruções de Comparação		sra	Rdst, Rsrc1, Imm5	\$a0..\$a3	\$4..\$7
		srav	Rdst, Rsrc1, Rsrc2	\$t0..\$t7	\$8..\$15
Salto Relativo (<i>Branch</i>) e Absoluto (<i>Jump</i>)		srl	Rdst, Rsrc1, Imm5	\$s0..\$s7	\$16..\$23
		srlv	Rdst, Rsrc1, Rsrc2	\$t8..\$t9	\$24..\$25
Manipulação de Exceções e Traps		Cálculo em Vírgula Flutuante		\$k0..\$k1	\$26..\$27
		add.p	FPdst, FPsrc1, FPsrc2	\$gp	\$28
Manipulação de Exceções e Traps		sub.p	FPdst, FPsrc1, FPsrc2	\$sp	\$29
		div.p	FPdst, FPsrc1, FPsrc2	\$fp	\$30
Manipulação de Exceções e Traps		mul.p	FPdst, FPsrc1, FPsrc2	\$ra	\$31
		neg.p	FPdst, FPsrc	Tabela II: Registos da FPU do MIPS e convenção de uso	
		abs.p	FPdst, FPsrc	Nome Lógico	Uso Convencionado
		cvt.d.s	FPdst, FPsrc	\$f0	Geral / valor de retorno das funções
Manipulação de Exceções e Traps		cvt.d.w	FPdst, FPsrc	\$f2..\$f10	Geral
		cvt.s.d	FPdst, FPsrc	\$f12..\$f14	Geral / passagem de parâmetros para funções
Manipulação de Exceções e Traps		cvt.s.w	FPdst, FPsrc	\$f16..\$f18	Geral
		cvt.w.d	FPdst, FPsrc	\$f20..\$f30	Geral - não podem ser alterados pelas funções
Manipulação de Exceções e Traps		cvt.w.s	FPdst, FPsrc	Rev 2024 - MBC, JLA, AO, LAU, ACP	
		c.eq.p	FPsrc1, FPsrc2		
		c.le.p	FPsrc1, FPsrc2		
Manipulação de Exceções e Traps		c.lt.p	FPsrc1, FPsrc2		
		break	n	Rev 2024 - MBC, JLA, AO, LAU, ACP	
		nop			
		eret			
Manipulação de Exceções e Traps		syscall			

Tabela III: Notação			
Imm	Valor imediato (constante) de 16 bits	addr	Endereço na forma Imm(Rsrc) = (Rsrc) + Imm
IMM	Valor imediato de 32 bits	B _k (Rsrc)	Byte índice k de Rsrc
Rsrc (1, 2)	Registo fonte (1 ou 2)	FPdst	Registo destino do coprocessador aritmético
(Rsrc)	Conteúdo de Rsrc	FPsrc (1, 2)	Registo fonte do coprocessador aritmético (1 ou 2)
Rdst	Registo destino	Src	Rsrc ou IMM
CReg	Registo do Coprocessador c1	c1	Coprocessador nº 1
sym	Endereço do símbolo (label) sym	Imm5	Valor imediato (constante) de 5 bits
Label	Endereço de uma instrução	.p	Precisão: substituir por .s ou .d

Tabela IV: System Calls do MARS			
Protótipo equivalent em C	\$v0	Parâmetros de entrada	Retorno
void print_int10(int value)	1	\$a0 = value	
void print_float(float value)	2	\$f12 = value	
void print_double(double value)	3	\$f12 = value	
void print_string(char *str)	4	\$a0 = str	
int read_int(void)	5		\$v0
float read_float(void)	6		\$f0
double read_double(void)	7		\$f0
void read_string(char *buf, int length)	8	\$a0=buf, \$a1=length	
void *sbrk(int amount)	9	\$a0 = amount	\$v0
void exit(void)	10		
void print_char(char value)	11	\$a0 = value	
char read_char(void)	12		\$v0
void print_int16(unsigned int value)	34	\$a0 = value	
void print_int2(unsigned int value)	35	\$a0 = value	
void print_intu10(unsigned int value)	36	\$a0 = value	

Tabela V - Directivas do Assembler	
Directivas	Descrição
Para controlo dos Segmentos	
.data [address]	Coloca os próximos itens no segmento de dados do utilizador (opcionalmente a partir de address).
.text [address]	Coloca os próximos itens no segmento de código do utilizador (opcionalmente a partir de address).
.kdata [address]	Coloca os próximos itens no segmento de dados do kernel (opcionalmente a partir de address).
.ktext [address]	Coloca os próximos itens no segmento de código do kernel (opcionalmente a partir de address).
Para criação de constantes e variáveis em memória:	
.ascii str	Armazena uma string em memória sem lhe acrescentar o terminador '\0'.
.asciiz str	Armazena uma string em memória acrescentando-lhe o terminador '\0'.
.space n	Reserva n bytes no segmento de dados, sem inicializar
.byte b ₁ , ..., b _n	Armazena as grandezas de 8 bits b ₁ , ..., b _n em sucessivos bytes de memória.
.word w ₁ , ..., w _n	Armazena as grandezas de 32 bits w ₁ , ..., w _n em sucessivas palavras de memória.
.float f ₁ , ..., f _n	Armazena f ₁ , ..., f _n em vírgula flutuante, precisão simples (32 bits) no seg. de dados.
.double d ₁ , ..., d _n	Armazena d ₁ , ..., d _n em vírgula flutuante, precisão dupla (64 bits) no seg. de dados.
.eqv label, valor	Substitui todas as ocorrências de label no programa por valor.
Para controlo do alinhamento:	
.align n	Alinha o próximo item num endereço múltiplo de 2 ⁿ .
Para referências externas:	
.globl sym	Declara que o símbolo sym é global e pode ser referenciado em outros ficheiros.
.extern sym size	Declara que o item associado a sym ocupa size bytes e é um símbolo global.
.include filename	Insere o conteúdo do ficheiro especificado (o nome do ficheiro é colocado entre aspas).